ISSUE: Cells may contain formulas, while we want only static data to expedite cleaning and for exporting.

TRADE OFFS:

Converting from formulas to static data makes our dataset inflexible to new data being added if there is data that depends on the data within the sheet. Examples of dependent data would be running totals or datestamps, which are made dead upon conversion to static data.

Since the goal as outlined is to clean for export, and not to be used for adding further data, this may be fine. Further, the cleaning paradigm clearly outlines that copies of the sheets will be made prior to cleaning, meaning that we do not risk much by making our data static.

SOLUTION:

The FormulaConversion.gs cleaning script will handle converting formulas into static data. It contains the following:

- convertFormulas(param {GoogleAppsScript.Spreadsheet.Sheet} sheet)
    - This is a cleaning function which handles converting all formulas in a sheet to their current calculated values.
- flagUnapprovedFormulas(param {GoogleAppsScript.Spreadsheet.Sheet} sheet)
    - Finds and logs formulas containing functions that are not approved
    - Each formula is checked for function names and compared against a list of approved functions. Any unapproved function is logged with the sheet name, cell address, and formula.

TESTING:

To confirm the conversion is safe, we wrote unit tests in FormulaConversionUnitTests.gs, that verify three things after conversion, these are stored within the FormulaConversionTesting.xlsx spreadsheet:

1. No formulas remain in the sheet
2. Cell formatting, how values are displayed, is unchanged (except for in the case of dates where we want to assign an explicit mm-dd-yyyy format)
3. Each cell's data type (text, number, date, true/false) is preserved, or correctly assigned once the formula is gone

Where applicable, every test, where applicable, applies all three checks along with a manual visual check of the sheet. The cases covered are the following:

- Formula results
    - A single formula returning each of: text, number, true/false, date, and a blank string

    - Multiple formulas returning each of those types
- Formula results displayed with common formats such as currency, percentages, dates, decimal places do not interfere and persist
- Special formulas
    - Formulas that return errors return a string describing the error
    - Formulas wrapped in error-handling functions
- A sheet with a single cell
- Formulas that reference other cells either above or below, for each result type above, as well as formulas that reference other formulas
- Formulas tied to changing data, like current date, confirming the value is frozen at the moment of conversion
- Formulas that produce more than plain values
    - Formulas that fill multiple cells at once

The cases below have not yet been thoroughly tested. We may not currently use these formulas, but they should be tested in the future to support scalability.

- Formulas that produce more than plain values
    - Formulas that create clickable links
    - Formulas that display images or in-cell charts
- External data: Formulas that pull data from outside the sheet

VALIDATION:

In order to support correct data handling, I also included a validation function with the purpose of checking for formulas that have not been tested. This works to flag potential errors by keeping an allowlist of formulas whose behavior had been tested. When a formula that is not on the accepted list is found the program will print out the following:

```
[SheetName] Unapproved FORMULANAME in cell: =FORMULANAME(...)
...
Needs manual verification: 1 formula(s) flagged across the spreadsheet
```

This allows for manual inspection in the case of an unexpected formula (such as the to be tested cases above).

The list of approved formulas are the following:

- Logical: AND, FALSE, IF, IFERROR, IFNA
- Math: SUM
- Statistical:
- Date: DATE, TODAY
- Operator: CONCAT, DIVIDE
- Array: ARRAYFORMULA
- Info: NA

Use: https://support.google.com/docs/table/25273?hl=en to continue adding more formula

ISSUE: Empty cells in sheets makes file size too large

SUBISSUE: What is an Empty Cell?

DEFINITION:

Empty, the way we think of it, may not be the same as the computer considers it. We likely want to remove any data that does not have significance, this includes a much broader range than just simply empty cells. The following cases, do not provide significant data and should be removed from our sheets:

- Formula cells returning “”
- Cells with whitespace
    - Non-breaking spaces
    - Zero-width characters
- Cells containing only apostrophes (with or without whitespace)
- Empty cells removed in the next stage, by deleting the rows and columns they occupy

SOLUTION:

1. Formula cells returning "": These are handled by the formula conversion stage. Cells containing such formulas are converted to static values, so each becomes a cell holding an empty string. Then the function trimWhiteSpaceCells() clears them.
    a. Call convertFormulas(). This is already done by cleanSheet.
2. Whitespace, invisible characters, and apostrophes: To treat these the same as empty cells, we convert them into truly empty cells. A single regular expression, BLANK_REGEX, matches cells made up entirely of whitespace and invisible characters (zero-width characters, joiners, fillers, and similar), and apostrophes. Cells that match are cleared. Cells with real content, including content with a stray invisible character, are left unchanged. A typed leading apostrophe is Sheets' text marker and is not part of the value.
    a. Call trimWhitespaceCells(). This is already done by emptyCellClean.
    b. Call SpreadsheetApp.flush() to commit changes. This is done by emptyCellClean.

TESTING:

To confirm that these cases were converted into empty cells, we wrote unit tests in EmptyCellsUnitTests.gs, the first five confirm that formulas returning blank strings, non-breaking spaces, zero-width characters, and blank apostrophe formatting are converted into empty cells.

VALIDATION:

We can validate the results with the function validateNoWhitespaceCells which collects the addresses of remaining cells that are whitespace only. This function uses an independent check from our cleaning regular expressions earlier, to avoid circularity in this validation.

SUBISSUE: Where are the Empty Cells?

CASES:

Case 1: Empty Row
First, let's discuss empty rows. These should, in all likelihood, always be removed. There are three positions our empty rows can be located in:

Subcase 1.1: Empty row above the valid data
Subase 1.2: Empty row centrally located within the valid data (Y-Bound)
Subase 1.3: Empty row below the valid data

Case 2: Empty Column
Second, we can have empty columns. These also may also be deleted unless the lack of entries gives significance to the data. These also can be located in three positions (left, X bounded, or right of the data) but also have the added complexity of being a headed column or not, leading to six new cases:

Subcase 2.1: Empty column left of the valid data
Subcase 2.2: Empty column centrally located within the valid data
Subcase 2.3: Empty column right of the valid data

Case 3: Internal Empty Cell
Finally, we want to discuss empty cells that are embedded within non-empty rows and non-empty columns. These must be cells that are both X and Y bounded, else they would fall into one of the above cases. There are two cases associated with these.

Subcase 3: Empty cell in a non-empty row and column

We can represent these cases on a cell level. The following table explains what cases might apply to an empty cell. If all the cases are handled, then no cell in that position will be empty after the cleaning:

	Left	X Bound	Right
Above	1.1 OR 2.1	1.1 OR 2.2	1.1 OR 2.3
Y Bound	1.2 OR 2.1	1.2 OR 2.2 OR 3	1.2 OR 2.3
Below	1.3 OR 2.1	1.3 OR 2.2	1.3 OR 2.3

We will want to remove the cases of fully empty rows and columns, as they will take up unnecessary and costly amount of space within our spreadsheet. Any cells belonging to the

categories X or Y bound should not be handled in this step as empty cells in these locations are meaningful as a lack of data. Techniques to handle this data loss might be checking the ways in which this data is retrieved for entry or web-scraping errors, or imputation in the case of eventual analysis.

TESTING:

In order to test that the cells are properly deleted we added a few more tests to EmptyCellsUnitTests.gs, one for each of the subcases in case one and two listed above.

ISSUE: Leading and trailing spaces within our dataset are causing mismatches and preventing accurate data comparisons.

DEFINITION:

Within the data there may be trailing spaces before or after the text, such as:

- “ Jane”
- “Jane “
- “ Jane “

All representing the same data point “Jane” in our dataset. In order to standardize the data and save on space, we will need to strip out these whitespaces.

SOLUTION:

TESTING:

VALIDATION: